

A DETAILED HIERARCHICAL DECOMPOSITION EXAMPLE

We provide the detailed illustration of the hierarchical decomposition process in MGDebugger, as shown in Figure 7, which has been simplified for illustration in Figure 1. For the original function “make_palindrome”, we decompose it into three minimal reusable subfunctions. And the relationships between subfunctions are naturally captured in the hierarchical structure based on the function calls. This hierarchical decomposition allows MGDebugger to systematically analyze and debug the code at different levels of granularity, leading to more effective identification and correction of bugs.

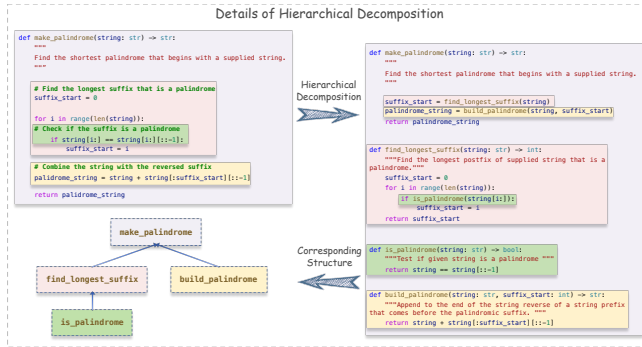


Figure 7: Detailed illustration of the hierarchical decomposition process in MGDebugger.

B HUMANEVALFIX DATASET

To access the ability of MGDebugger in debugging code with different types of bugs, we use the HumanEvalFix dataset [42], which consists of 164 buggy functions across six programming languages, each provided with solutions and unit tests. For our experiments, we focus on the Python subset of the dataset. The buggy functions are categorized into six types of bugs: value misuse, missing logic, excess logic, operator misuse, variable misuse, and function misuse. Table 6 shows the distribution and explanations of these bug types within the HumanEvalFix dataset.

Table 6: Distribution and explanations of bugs in the HumanEvalFix dataset.

Bug Category	Explanation	Count
Value Misuse	An incorrect value is used	44
Missing Logic	Misses code needed to solve the problem	33
Excess Logic	Contains excess code leading to mistakes	31
Operator Misuse	An incorrect operator is used	25
Variable Misuse	An incorrect variable is used	23
Function Misuse	An incorrect function is used	8

C PROMPT TEMPLATES FOR MGDEBUGGER

We provide prompt templates for the three main components of MGDebugger: hierarchical decomposition, test case generation,

and sub-function debugging with LLM-simulated execution. These prompts are designed to guide the language model in generating the desired outputs for each step of the debugging process. They have been slightly modified for clarity and brevity, please refer to our codes if you need the exact prompt templates⁵.

C.1 Prompt for Hierarchical Decomposition

Prompt Template for Hierarchical Decomposition

SYSTEM_PROMPT:

You are an AI assistant specialized in refactoring Python code into a tree-style hierarchical structure.

USER_PROMPT:

Convert the following Python code into a tree-style hierarchical structure with multiple levels of sub-functions. Each significant step or logical block should be its own function, and functions can call other sub-functions. Ensure that the main function calls these sub-functions in the correct order, creating a tree-like structure.

Original Code:

{code}

Instruction:

Please first analyze the codes step by step, and then provide the converted code in a Python code block. When providing the final converted code, make sure to include all the functions in a flattened format, where each function is defined separately.

C.2 Prompt for Test Case Generation

Prompt for Test Case Generation

SYSTEM_PROMPT:

You are an AI assistant specialized in analyzing Python functions and generating test cases.

USER_PROMPT:

Full Code:

{full_code}

Public Test Cases for the Main Function:

{public_test_cases}

Instruction:

Please analyze how the {function_name} function is used within the main function and how it contributes to the expected outputs in the gold test cases. For each test case, you should analyze step-by-step based on both the input and the expected output of the main function, and then provide the corresponding input and expected output for the {function_name} function. Ensure that the generated test cases are consistent with the behavior expected in the public test cases.

⁵<https://anonymous.4open.science/r/MGDebugger-B388>

C.3 Prompt for Debugging Subfunction

Prompt for Debugging Subfunction

SYSTEM_PROMPT:
You are an AI assistant helping to debug Python functions.
USER_PROMPT:
Debug the following Python function. The function is not passing all test cases. Analyze the code, identify the bug, and provide a fixed version of the function.
Function Code:
{function_code}
Test Case Results:
{test_case_results}
Instruction:
Please try to work as a Python interpreter to execute the code step-by-step. Identify the change of each variable as you “run” the code line-by-line. Based on the execution trace, try to identify the bug and provide the final fixed code in a Python code block.

D BASELINE PROMPTS

This section provides the detailed prompts used for each of the baseline approaches compared with MGDebugger. For LDB [70] and Reflexion [49], we use their official codebases. For Self-Edit [64] and Self-Debugging [9], we closely follow the implementation details and procedures described in their respective papers to replicate the methods. The detailed codes of the baselines are also available under the `baselines` folder of our repository⁶.

D.1 No-Debugging Baseline

No-debugging approach generates seed buggy solutions for other debugging baselines to repair. We follow the codes from LDB [70] to set up this baseline.

No-Debugging Baseline

SYSTEM_PROMPT:
You are an expert programming assistant.
USER_PROMPT:
Complete the following task in Python. Please respond with code only.
{problem_description}

D.2 Simple Feedback Baseline

This baseline collects minimal feedback by providing only the buggy code and basic error information to the model. We implement this baseline following [9].

Simple Feedback Baseline

SYSTEM_PROMPT:
You are an AI assistant specialized in debugging Python code.
USER_PROMPT:
The following Python code is buggy. Please fix it and provide the corrected version. Function name: '{entry_point}'
Buggy code: {buggy_code}
Please fix the code and provide ONLY the final fixed code in a Python code block (“python ...”). Make sure to include the entire function, including the function signature.

D.3 Self-Edit Baseline

This baseline collects feedback through unit test execution results and expected vs. actual outputs for failing test cases. We implement this baseline following the methodology described in [64], where we execute the generated code on example test cases and embed the results as comments.

Self-Edit Baseline

SYSTEM_PROMPT:
You are an AI assistant specialized in debugging Python code.
USER_PROMPT:
The following Python code is buggy. Please fix it based on the provided test cases and feedback. Function name: '{entry_point}'
Buggy code: {buggy_code}
Test cases: {test_cases}
Unit test feedback: {execution_traces}
Please fix the code and provide ONLY the final fixed code in a Python code block (“python ...”). Make sure to include the entire function, including the function signature.

D.4 Self-Debugging (Explanation) Baseline

This baseline collects comprehensive feedback including complete unit test execution results, detailed execution traces, and error messages. We implement this baseline following the approach described in [9], implementing the generation, explanation, and feedback steps, using unit tests to produce feedback and leveraging LLMs for explanation.

⁶<https://anonymous.4open.science/r/MGDebugger-B388>

Self-Debugging (Explanation)**SYSTEM_PROMPT:**

You are an AI assistant specialized in debugging Python code.

USER_PROMPT:

The following Python code is buggy. Please analyze the code step by step, explain what might be wrong, and then provide the corrected version. Function name: '{entry_point}'

Buggy code: {buggy_code}

Test cases: {test_cases}

Unit test feedback: {execution_traces}

Please: 1. First, explain what you think is wrong with the code 2. Then provide the fixed code in a Python code block ("python ...") Make sure to include the entire function, including the function signature.

D.5 Self-Debugging (Trace) Baseline

This baseline collects feedback through step-by-step execution traces showing intermediate variable values and program execution flow. This variant follows the trace-based debugging approach described in [9].

Self-Debugging (Trace)**SYSTEM_PROMPT:**

You are an AI assistant specialized in debugging Python code.

USER_PROMPT:

The following Python code is buggy. Please trace through the execution step by step, identify the issue, and provide the corrected version. Function name: '{entry_point}'

Buggy code: {buggy_code}

Test cases: {test_cases}

Execution traces: {execution_traces}

Please: 1. Trace through the code execution step by step 2. Identify where the bug occurs 3. Provide the fixed code in a Python code block ("python ...") Make sure to include the entire function, including the function signature.

D.6 LDB Line-level Baseline

The LDB baseline uses line-level granularity for debugging. We use the official codebase of LDB [70] with line-level granularity settings. The prompts follow the original LDB paper implementation with external feedback mechanisms. This baseline collects feedback by tracking execution at individual code lines and collecting variable states after each line execution.

LDB Line-level**SYSTEM_PROMPT:**

You are an expert programming assistant specialized in debugging Python code.

USER_PROMPT:

Debug the following Python code at the line level. Identify and fix bugs line by line. Function name: '{entry_point}'

Buggy code: {buggy_code}

Test feedback: {test_feedback}

Please provide the corrected code with line-level fixes.

D.7 LDB Block-level Baseline

The LDB baseline uses block-level granularity for debugging. We use the official LDB codebase [70] with block-level granularity settings. This baseline collects feedback by segmenting programs into basic blocks and tracking intermediate variable values after each block.

LDB Block-level**SYSTEM_PROMPT:**

You are an expert programming assistant specialized in debugging Python code.

USER_PROMPT:

Debug the following Python code at the block level. Identify and fix bugs in code blocks. Function name: '{entry_point}'

Buggy code: {buggy_code}

Test feedback: {test_feedback}

Please provide the corrected code with block-level fixes.

D.8 LDB Function-level Baseline

The LDB baseline uses function-level granularity for debugging. We use the official LDB codebase [70] with function-level granularity settings. This baseline collects feedback through function-level execution information and return values.

LDB Function-level**SYSTEM_PROMPT:**

You are an expert programming assistant specialized in debugging Python code.

USER_PROMPT:

Debug the following Python code at the function level. Analyze the entire function and provide fixes. Function name: '{entry_point}'

Buggy code: {buggy_code}

Test feedback: {test_feedback}

Please provide the corrected function.

D.9 Reflexion

The Reflexion baseline incorporates self-reflection mechanisms as described in the original Reflexion paper. We use the official Reflexion codebase [49].

This baseline collects feedback through unit test execution results and self-generated reflective text stored in episodic memory for improved decision-making.

Reflexion

SYSTEM_PROMPT:

You are a Python writing assistant. You will be given your previous implementation of a function, a series of unit tests results, and your self-reflection on your previous implementation. Apply the necessary changes below by responding only with the improved body of the function. Do not include the signature in your response. The first line of your response should have 4 spaces of indentation so that it fits syntactically with the user provided signature.

USER_PROMPT:

Previous implementation: {previous_implementation}

Unit test results: {unit_test_results}

Self-reflection: {self_reflection}

Please provide the improved function body based on the above information.

E INDIVIDUAL COMPONENT ANALYSIS

This section provides manual evaluation and error analysis of MGDebugger's core components (decomposition, test generation, and execution simulation) based on a random subset of 10 examples from DeepSeek-Coder-v2 on HumanEval (7 successfully debugged ones and 3 failed ones). We will include more detailed analysis on a larger set of data, and also inspect on quality and adequacy of LLM-generated test cases in the final version.

E.1 Code Decomposition Effectiveness

Our manual verification shows that all 10 decomposed versions are functionally equivalent to the original functions. This demonstrates the high quality of our LLM-based decomposition approach. Since the overall codes have been provided, it's less likely that the model include new bugs when transforming the code into subfunctions.

E.2 Test Case Generation Accuracy

In MGDebugger, the model is instructed to strictly follow the public test cases from the original problems when generating test cases for each subfunction. We evaluated test case generation accuracy by manually verifying the correctness of the generated test cases. The results show that LLM-generated test cases are correct in 142/151 = 94.0% of cases, which means that only 6% of the test cases are incorrect.

We acknowledge that the quality and adequacy of the generated test cases are also important for the effectiveness of the debugging process. If provided testcases with more coverage and more diverse cases, the performance of MGDebugger could be further improved

since more bugs could be found and more information could be provided to the model. We regard this as promising directions for future research. And we will also include more detailed analysis on the coverage, mutation score, and other metrics on the test cases in the final version.

E.3 Simulated Execution Analysis

We conducted a manual verification on the simulated execution results by comparing to the real execution results at each step. The LLM correctly predicted 124 out of 129 breakpoints (96.1% accuracy), with failures occurring only in cases requiring complex multi-step reasoning. The high accuracy of simulated execution aligns well with existing studies [43, 68]. And the superior performance of simulated execution is also supported by our ablation study, where the model will suffer a 15.0% loss of RSR on HumanEval. Interestingly, we found that if we manually correct the prediction errors based on real execution, one of three failed debugging cases can be further fixed by MGDebugger. This suggests that improving the quality of simulated execution could further improve the performance of MGDebugger.

E.4 Conclusion

According to our manual analysis, the high quality of each component in MGDebugger contributes to the overall effectiveness of our approach, as demonstrated by the ablation results. This suggests our approach is robust and that further improvements in component quality, including enhancing test case generation and adequacy, could lead to even better results. We regard this as promising directions for future research and will include more detailed analysis on a larger set of examples in the final version.